

WMO WIS2 Notification Message Encoding

World Meteorological Organization

Date: 2026-06-15

Version: 1.3.0

Document status: STABLE

Revision history: see [Annex E](#)

Document location: <https://wmo-im.github.io/wis2-notification-message/standard/wis2-notification-message-STABLE.html>

WMO publication location: <https://library.wmo.int/idurl/4/68731>

Standing Committee on Information Management and Technology (SC-IMT)^[1]

Commission for Observation, Infrastructure and Information Systems (INFCOM)^[2]

Copyright © 2024-2026 World Meteorological Organization (WMO)

Table of Contents

1. Scope	5
2. Conformance	6
3. References	7
4. Terms and definitions	8
4.1. Abbreviated terms	8
5. Conventions	10
5.1. Identifiers	10
5.2. Examples	10
5.3. Schemas	10
5.4. Schema representation	10
5.4.1. Properties	10
6. Introduction	11
6.1. Motivation	11
6.2. Scenarios	11
6.2.1. Publish, Subscribe, Download	11
6.2.2. Replay API workflow	11
7. WIS2 Notification Message	12
1. Requirements Class "Core"	12
1.1 Overview	12
1.2 Message size	14
1.3 GeoJSON compliance	14
1.4 Identifier	14
1.5 Conformance	15
1.6 Version	15
1.7 Geometry	16
1.8 Properties / Publication time	17
1.9 Properties / Data identification	19
1.10 Properties / Metadata identification	19
1.11 Properties / Producer	20
1.12 Properties / Global Cache	20
1.13 Properties / Temporal description	21
1.14 Properties / Cache	22
1.15 Properties / Integrity	22
1.16 Properties / Content	23
1.17 Links	24
1.18 Additional properties	26
2. WIS2 notification message resources	28
2.1 WMO Codes Registry	28

2.2 WMO schemas server	28
Annex A: Conformance Class Abstract Test Suite (Normative)	29
Conformance Class: Core	29
Message size	29
Validation	29
Identifier	30
Conformance	30
Version	30
Geometry	31
Properties / Publication Time	31
Properties / Data Identification	32
Properties / Temporal description	32
Properties / Global Cache	32
Links	32
Annex B: Schemas (Normative)	34
WIS2 Notification Message Schema	34
Annex C: Examples (Informative)	39
WIS2 Notification Message Examples	39
Annex D: Bibliography	43
Annex E: Revision History	44

i. Abstract

WIS2 real-time data sharing is based on a message queuing protocol (MQP) supporting a publication/subscription (PubSub) mechanism. A user can subscribe to an MQP broker to receive real-time notifications of the existence of new data. The notification message received from the MQP broker contains a URL to download the data. In addition, the MQP broker offers a range of topics organised in a hierarchy. The users can select their topics of interest and subscribe to them to receive notifications and download data relevant to their work.

The standard notification message format ensures that the WIS2 ecosystem (data publisher, data user, and global services) is a robust, effective, and unified exchange platform for weather, climate, and water data.

This document defines the content, structure, and encoding for the WIS2 Notification Message Encoding. This standard is an extension of the OGC API - Features standard ^[3]. WIS2 Notification Message documents are provided as MQP payloads by WIS2 nodes, Global Broker services, as well as Replay API services (optional OGC API - Features services for data notifications).

WIS2 Notification Message documents shall be encoded in GeoJSON (RFC7946 ^[4]) as defined in this specification and shall be made available as MQP payloads. Additionally, they can be provisioned as defined by OGC API - Features.

Weather/climate/water data is by nature geospatial and temporal. The W3C Data on the Web Best Practices ^[5] and Spatial Data on the Web Best Practices ^[6] publications provide guidelines on how to best enable spatiotemporal data to lower the barrier for users, search engine optimization, and linked data. This also aligns with the FAIR data principles (Findable, Accessible, Interoperable, Reusable) ^[7].

ii. Keywords

The following are keywords to be used by search engines and document catalogues.

wmo, wis 2.0, weather, climate, water, metadata, pubsub, mqp, message queuing protocol, GeoJSON

iii. Security Considerations

Based on the WMO Unified Data Policy for the International Exchange of Earth System Data (Resolution 1 (Cg-Ext(2021)) ^[8], exchanged data are classified as **core** or **recommended**. Core data is considered fully open and unrestricted with no security considerations. Recommended data may have access control defined.

No security considerations have been made for this standard.

Chapter 1. Scope

This document defines the content, structure, and encoding of a notification message published as part of a WIS2 Global Broker or a Replay API service.

The WIS2 Notification Message Encoding standard defined herein is an extension of the International Standard *GeoJSON*.

WIS2 Notification Message documents shall be encoded as GeoJSON as defined in *OGC API - Features - Part 1: Core*.

The primary purpose of WIS2 Notification Messages are to notify subscribers or clients of new data being published.

This specification defines the conformance requirements for the WIS2 Notification Message Encoding. Annex A defines the abstract test suite.

[1] <https://community.wmo.int/governance/commission-membership/commission-observation-infrastructures-and-information-systems-infcom/commission-infrastructure-officers/infcom-management-group/standing-committee-information-management-and-technology-sc-int>

[2] <https://community.wmo.int/governance/commission-membership/infcom>

[3] <https://ogcapi.ogc.org/features>

[4] <https://datatracker.ietf.org/doc/html/rfc7946>

[5] <https://www.w3.org/TR/dwbp>

[6] <https://www.w3.org/TR/sdw-bp>

[7] https://en.wikipedia.org/wiki/FAIR_data

[8] https://library.wmo.int/doc_num.php?explnum_id=11113#page=9

Chapter 2. Conformance

Conformance with this standard shall be checked using the tests specified in Annex A (normative) of this document.

GeoJSON provides an encoding for encoding geographic features (geometry, attributes). This standard is an extension of *GeoJSON*.

Data providers are required to comply with all conformance classes of this specification in support of providing MQP services.

WMO shall publish guidance material to assist data providers in constructing WIS2 Notification Messages.

This standard identifies one Requirements Class which defines the functional requirements.

The mandatory Requirements Class for this specification is:

- "WIS2 Notification Message Encoding Core"

Chapter 3. References

- OGC: OGC 17-069r4, OGC API - Features - Part 1: Core 1.0.1 (2022) ^[1]
- IETF: RFC-7946 The GeoJSON Format (2016) ^[2]
- IETF: RFC-8259 The JavaScript Object Notation (JSON) Data Interchange Format (2017) ^[3]
- IETF: RFC 3339: Date and Time on the Internet: Timestamps (2002) ^[4]
- IETF: RFC 4122: A Universally Unique Identifier (UUID) URN Namespace (2005) ^[5]
- W3C/OGC: Spatial Data on the Web Best Practices, W3C Working Group Note (2017) ^[6]
- W3C: Data on the Web Best Practices, W3C Recommendation (2017) ^[7]
- W3C: Data Catalog Vocabulary, W3C Recommendation (2014) ^[8]
- IANA: Link Relation Types (2020) ^[9]
- IETF: JSON Schema (2022) ^[10]
- WMO: WIS2 Topic Hierarchy (2022) ^[11]

[1] <https://docs.openeospatial.org/is/17-069r4/17-069r4.html>

[2] <https://datatracker.ietf.org/doc/html/rfc7946>

[3] <https://datatracker.ietf.org/doc/html/rfc8259>

[4] <https://datatracker.ietf.org/doc/html/rfc3339>

[5] <https://datatracker.ietf.org/doc/html/rfc4122>

[6] <https://www.w3.org/TR/sdw-bp>

[7] <https://www.w3.org/TR/dwbp>

[8] <https://www.w3.org/TR/vocab-dcat>

[9] <https://www.iana.org/assignments/link-relations/link-relations.xml>

[10] <https://json-schema.org>

[11] <https://github.com/wmo-im/wis2-topic-hierarchy>

Chapter 4. Terms and definitions

This document uses the terms defined in [OGC Policy Directive 49](#), which is based on the ISO/IEC Directives, Part 2, Rules for the structure and drafting of International Standards. In particular, the word “shall” (not “must”) is the verb form used to indicate a requirement to be strictly followed to conform to this Standard and OGC documents do not use the equivalent phrases in the ISO/IEC Directives, Part 2.

This document also uses terms defined in the OGC Standard for Modular specifications ([OGC 08-131r3](#)), also known as the 'ModSpec'. The definitions of terms such as standard, specification, requirement, and conformance test are provided in the ModSpec.

The following additional terms and definitions also apply.

4.1. Abbreviated terms

Table 1. Symbols and abbreviated terms

Abbreviation	Term
API	Application Programming Interface
DCPC	Data Collection and Production Centres
GDC	Global Discovery Catalogue
GIS	Geographic Information System
GISC	Global Information System Centre
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IANA	Internet Assigned Numbers Authority
IETF	Internet Engineering Task Force
ISO	International Organization for Standardization
JSON	JavaScript Object Notation
MQP	Message Queuing Protocol
MQTT	Message Queuing Telemetry Transport
NC	National Centre
NWP	Numerical Weather Prediction
OGC	Open Geospatial Consortium
PubSub	Publish / Subscribe
URI	Uniform Resource Identifier
URL	Uniform Resource Locator
UUID	Universally Unique Identifier

Abbreviation	Term
W3C	World Wide Web Consortium
WCMP	WMO Core Metadata Profile
WIS	WMO Information System
WMO	World Meteorological Organization
WNM	WIS2 notification message

Chapter 5. Conventions

This section provides details and examples for any conventions used in the document. Examples of conventions are symbols, abbreviations, use of JSON schema, or special notes regarding how to read the document.

5.1. Identifiers

The normative provisions in this Standard are denoted by the URI:

<http://wis.wmo.int/spec/wnm/1>

All requirements and conformance tests that appear in this document are denoted by partial URIs which are relative to this base.

5.2. Examples

Examples provided in this specification are encoded as GeoJSON.

Complete examples can be found at <https://schemas.wmo.int/wnm/1.1.0/examples>

5.3. Schemas

The WIS2 Notification Message schema can be found at <https://schemas.wmo.int/wnm/1.1.0/schemas/wis2-notification-message-bundled.json>

5.4. Schema representation

JSON Schema ^[1] objects are used throughout this standard to define the structure of metadata records. These schema objects are also typically represented using YAML ^[2]. YAML is a superset of JSON, and in this standard are regarded as equivalent.

Metadata record instances are always defined as JSON.

5.4.1. Properties

A JSON **property** represents a key-value pair, where the key is the name of the property and the value is a standard JSON data type.

```
"myPropertyName": "test123"
```

[1] <https://json-schema.org>

[2] <https://en.wikipedia.org/wiki/YAML>

Chapter 6. Introduction

A WIS2 Notification Message (WNM) provides descriptive information about data or metadata made available through WIS 2.0. The standard notification message format ensures that the WIS2 ecosystem (data publisher, data user, and global services) is a robust, effective, and unified data exchange platform for weather, climate, and water.

WIS2 Notification Message documents are provided as MQP payloads by WIS2 nodes, Global Broker services, as well as Replay API services.

A WNM provides notification metadata about data or metadata published to WIS2, including when the data was published, its spatial and temporal characteristics, and where to access and download.

6.1. Motivation

MQP brokers provide "push" based services, supporting event-driven workflows, maximizing efficient use of network, bandwidth, and rapid response to time-sensitive events (e.g. severe weather events). However, MQP brokers do not specify a payload encoding. Using GeoJSON as a baseline for this specification provides broad interoperability, lowering the barrier and extending the reach of data in the WIS2 ecosystem and beyond.

6.2. Scenarios

The following scenarios are useful in understanding the drivers and principles that were used in the development of this specification:

6.2.1. Publish, Subscribe, Download

Event driven (PubSub) workflow involves a client connecting to a MQP broker, subscribing to one or more topics and receiving relevant notifications. Notifications provide information to ensure data integrity and spatiotemporal extents of a data granule.

6.2.2. Replay API workflow

API workflow involves a client connecting to an OGC API - Features services in order to query for messages in the past using spatial, temporal, or attribute predicates. A Replay API workflow is valuable in situations where connections to MQP brokers may drop, thereby allowing a given subscriber to gather past messages.

Chapter 7. WIS2 Notification Message

Note: This section of this working draft document is the same as Appendix E in the [Manual on the WMO Information System](#) (WMO-No. 1060), Volume II as approved by EC-78.

The WIS2 Notification Message (WNM) is an extension of the Open Geospatial Consortium *OGC API - Features* standard and shall be encoded in GeoJSON. The normative provisions in the WIS2 Notification Message are denoted by the base URI (<http://wis.wmo.int/spec/wnm/1>) and requirements are denoted by partial URIs relative to this base. Property names, values and examples are represented with **shaded** text in this document.

1. Requirements Class "Core"

URI	http://wis.wmo.int/spec/wnm/1/req/core
Target type	Notification metadata
Dependency	The JavaScript Object Notation (JSON) Data Interchange Format (IETF RFC8259 (2017))
Dependency	JSON Schema (2022)
Dependency	The GeoJSON Format (IETF: RFC-7946 (2016))
Dependency	OGC API – Features – Part 1: Core corrigendum (OGC: OGC 17-069r)
Preconditions	The record conforms to GeoJSON (RFC7946)

1.1 Overview

The table below provides an overview of the set of properties that may be included in a WNM.

Table 2. WNM core properties

Property	Requirement	Description
id	Required	A universally unique identifier (UUID) of the message (see 1.4 Identifier)
type	Required	A fixed value denoting the record as a GeoJSON Feature (see 1.3 GeoJSON compliance)
conformsTo	Required (one of conformsTo or version)	The version of WNM associated that the record conforms to (see 1.5 Conformance).
version	Required (one of conformsTo or version)	DEPRECATED Please use conformsTo (see 1.5 Conformance)

Property	Requirement	Description
<code>geometry</code>	Required	Geospatial location associated with the data or metadata (see 1.7 Geometry)
<code>properties.pubtime</code>	Required	The date and time when the notification was published, in RFC3339 format, Coordinated Universal Time (UTC) (see 1.8 Properties / Publication time)
<code>properties.data_id</code>	Required	Unique identifier of the data as defined by the data producer (see 1.9 Properties / Data identification)
<code>properties.metadata_id</code>	Optional	Identifier for associated discovery metadata record to which the notification applies (see 1.10 Properties / Metadata identification)
<code>properties.producer</code>	Optional	Identifies the provider that initially captured and processed the source data, in support of data distribution on behalf of other Members (see 1.11 Properties / Producer)
<code>properties.global-cache</code>	Optional	Centre identifier of Global Cache Service producing the notification (see 1.12 Properties / Global Cache)
<code>properties.datetime</code>	Optional	Identifies the reference date and time of the data instance to which the notification is relayed, in RFC3339 format, UTC (see 1.13 Properties / Temporal description)
<code>properties.start_datetime</code>	Optional	Identifies the start date and time of the data being published, in RFC3339 format (see 1.13 Properties / Temporal description)
<code>properties.end_datetime</code>	Optional	Identifies the end date and time of the data being published, in RFC3339 format (see 1.13 Properties / Temporal description)

Property	Requirement	Description
<code>properties.cache</code>	Optional	Indicates whether the data in the notification should be cached (if not specified, the default value is <code>true</code>) (see 1.14 Properties / Cache)
<code>properties.integrity</code>	Optional	Specifies a checksum to be applied to the data to ensure that the download is accurate (see 1.15 Properties / Integrity)
<code>properties.content</code>	Optional	Used to embed small products inline within the message (see 1.16 Properties / Content)
<code>links</code>	Required	Online linkages for data retrieval or additional resources associated with the dataset (see 1.17 Links)

1.2 Message size

The WIS2 Notification Message allows for the transmission of messages in a compact manner and includes the ability to embed content inline as required (see [1.16 Properties / Content](#)).

Requirement 1	<code>/req/core/message_size</code>
A	A WNM message SHALL NOT exceed 8 192 bytes.

1.3 GeoJSON compliance

The WIS2 Notification Message schema is based on *GeoJSON* (RFC7946) and its associated information model. Compliant messages are therefore compliant with *GeoJSON*.

Requirement 2	<code>/req/core/validation</code>
A	Each WNM SHALL validate without error against the WNM schema.
B	Each WNM SHALL provide <code>id</code> , <code>type</code> , <code>geometry</code> and <code>properties</code> properties for GeoJSON compliance.
C	Each WNM record <code>type</code> property SHALL be set to a fixed value of <code>Feature</code> for GeoJSON compliance.

1.4 Identifier

A universally unique identifier of the message is generated by the originator of the message, using the UUID standard ([RFC4122](#)). It provides the anti-loop feature that is needed to ensure that the message will be seen once by all Global Brokers. It remains the same throughout the lifetime of the

message in the WIS2 ecosystem.

The `data_id` is retained to ensure traceability and consistency of the same resource (see [1.9 Properties / Data identification](#)).

Example:

```
"id": "31e9d66a-cd83-4174-9429-b932f1abe1be"
```

Requirement 3	/req/core/identifier
A	The <code>id</code> property SHALL be a Universally Unique Identifier (UUID).

1.5 Conformance

The `conformsTo` property identifies the version of the WNM standard to which the notification conforms. Conformance identification is valuable for version detection and handling of content.

NOTE that `conformsTo` replaces `version`. For the deprecation period, only one (but not both) of `conformsTo` or `version` are permitted.

Example:

```
"conformsTo": [  
  "http://wis.wmo.int/spec/wnm/1/conf/core"  
]
```

Requirement 4	/req/core/conformance
A	A WNM SHALL provide information on conformance via the OAFeat <code>conformsTo</code> property.
B	The <code>conformsTo</code> property SHALL advertise conformance to WNM.

1.6 Version

DEPRECATED Please use `conformsTo`.

The `version` property provides the version of WNM that the message conforms to.

NOTE that `conformsTo` replaces `version`. For the deprecation period, only one (but not both) of `conformsTo` or `version` are permitted.

Requirement 5	/req/core/version
A	A WNM SHALL provide information on version conformance via the <code>version</code> property.

B

The **version** property SHALL be fixed to **v04** for this version of the specification.

1.7 Geometry

The type of geometry in a notification message may be **Point** or **Polygon**. It can also be type **null** if the geometry cannot be derived.

Example: Point

```
{
  ...
  "geometry": {
    "type": "Point",
    "coordinates": [
      6.146255135536194,
      46.223296618227444
    ]
  }
  ...
}
```

Example: Point with elevation

```
{
  ...
  "geometry": {
    "type": "Point",
    "coordinates": [
      6.146255135536194,
      46.223296618227444,
      392
    ]
  }
  ...
}
```

Example: Polygon

```
{
  ...
  "geometry": {
    "type": "Polygon",
    "coordinates": [[
      [-7.75,40.43],
      [-7.75,78.46],
      [71.91,78.46],
      [71.91,40.43],
      [-7.75,40.43]
    ]
  }
}
```

```

    ]]
  }
  ...
}

```

Example: null

```

{
  ...
  "geometry": null
  ...
}

```

Requirement 6	/req/core/geometry
A	A WNM record SHALL provide one geometry property to convey the geospatial properties of a notification using a geographic coordinate reference system (World Geodetic System 1984 [WGS 84]) with longitude and latitude decimal degree units.
B	The geometry property SHALL only provide one of a Point or Polygon geometry, or a null value when a geometry value is unknown or cannot be determined.

Permission 1	/per/core/geometry
A	The geometry property MAY provide a third element (height) as per clause 4 of the GeoJSON specification.

1.8 Properties / Publication time

The **pubtime** property identifies the date/time when the notification was posted or published by the originator. The date/time is encoded in RFC3339 format, using UTC (Z).

The publication date/time is critical for subscribers. It prevents message loss by providing awareness among subscribers about how far behind the publisher they may be.

The **pubtime** property is also valuable for change detection as part of updates and deletion notifications.

Ensuring that **pubtime** is properly managed for updates and deletions is important for data and metadata download workflows. For example, an out-of-date **pubtime** can lead to errors for clients when managing updates or deletions in their local copies of data. An update with a newer **pubtime**, identical **data_id** and a link with **rel=update** indicates a newer version of the data or metadata.

Example:

```

"properties": {
  ...
  "pubtime": "2022-03-20T04:50:18.314854383Z"
}

```

```
...
}
```

Example: using **pubtime**, **data_id** and a link **rel=update** to provide a data/metadata notification: initial notification

```
"properties": {
  ...
  "pubtime": "2022-03-20T04:50:18.314854383Z"
  "data_id": "wis2/ma-marocmeteo/data/core/weather/surface-based-
observations/synop/WIGOS_0-504-1-60288_20240210T130000"
  ...
},
"links": [{
  "href": "https://example.org/data/4Pubsub/92c557ef-d28e-4713-91af-
2e2e7be6f8ab.bufr4",
  "rel": "canonical",
  "type": "application/bufr"
}]
```

Example: using **pubtime**, **data_id** and a link **rel=update** to provide a data/metadata notification: update notification

```
"properties": {
  ...
  "pubtime": "2022-03-21T03:14:44.234524524Z"
  "data_id": "wis2/ma-marocmeteo/data/core/weather/surface-based-
observations/synop/WIGOS_0-504-1-60288_20240210T130000"
  ...
},
"links": [{
  "href": "https://example.org/data/4Pubsub/61156c54-dadf-4cb3-8b04-
1fdcb472036d.bufr4",
  "rel": "update",
  "type": "application/bufr"
}]
```

Requirement 7	/req/core/pubtime
A	A WNM SHALL provide a properties.pubtime property.
B	The properties.pubtime property SHALL be in RFC3339 format.
C	The properties.pubtime property SHALL be in UTC.
D	The properties.pubtime property SHALL be set to the current time by the original publisher of the notification.
E	The properties.pubtime property SHALL be set to the current time also for notifications about updates or deletions.

F	The <code>properties.pubtime</code> property SHALL NOT be modified by any intermediaries.
---	---

1.9 Properties / Data identification

The `data_id` property uniquely identifies the data described by the notification and is defined by the data producer. A data producer may use an identification scheme of their choice.

Example:

```
"properties": {
  ...
  "data_id": "wis2/ma-marocmeteo/data/core/weather/surface-based-
observations/synop/WIGOS_0-504-1-60288_20240210T130000"
  ...
}
```

Requirement 8	/req/core/data_id
A	A WNM SHALL provide a <code>properties.data_id</code> property.
B	The <code>properties.data_id</code> property SHALL be unique within the scope of the relevant dataset.

Recommendation 1	/rec/core/data_id
A	The <code>properties.data_id</code> property SHOULD NOT use an opaque id. It should be encoded with meaningful values to support client-side filtering.

Permission 2	/per/core/data_id
A	The <code>properties.data_id</code> property MAY contain a valid WIS2 topic, without the channel and version.

1.10 Properties / Metadata identification

The `metadata_id` property uniquely identifies the associated discovery metadata record. This property is an important linkage between a WMO Core Metadata Profile (WCMP) dataset discovery metadata record and the related data notifications. The inclusion of this property allows a subscriber to consult additional documentation of the dataset and understand the access control applied to the data.

Example:

```
"properties": {
  ...
  "metadata_id": "urn:wmo:md:ca-eccc-msc:observations.swob"
  ...
}
```

```
}
```

Recommendation 2	/rec/core/metadata_id
A	A WNM SHOULD provide a <code>properties.metadata_id</code> property, which is a string, identifying the associated WCMP dataset discovery metadata record. See requirement for metadata identification in WCMP (Appendix F, 1.4 Identifier).

Permission 3	/per/core/metadata_id
A	A WNM MAY provide additional identifiers to discovery metadata records in non-WIS2 catalogue systems as a link object with (<code>rel=related</code> or <code>rel=up</code>).

1.11 Properties / Producer

The `producer` property identifies the provider that initially captured and processed the source data, in support of data distribution on behalf of other Members.

Example:

```
"properties": {  
  ...  
  "producer": "fra"  
  ...  
}
```

Recommendation 3	/rec/core/producer
A	A WNM SHOULD provide a <code>properties.producer</code> property when publishing data on behalf of other Members.

1.12 Properties / Global Cache

The `global-cache` property identifies the centre identifier of the Global Cache Service producing the message.

Example:

```
"properties": {  
  ...  
  "global-cache": "de-dwd-global-cache"  
  ...  
}
```

Requirement 9	/req/core/global-cache
---------------	------------------------

A	When a notification is produced by a Global Cache Service, a WNM SHALL provide a <code>properties.global-cache</code> property.
---	---

1.13 Properties / Temporal description

The `datetime` property identifies the date and time of the data (for example, when a measurement was observed).

The `start_datetime` and `end_datetime` properties identify a temporal extent (for example, the start and end times of an NWP forecasting period). All dates and times are encoded in RFC3339 format using UTC (Z). A `null` value can also be used if a temporal description of the data cannot be derived.

Example: Temporal instant

```
"properties": {
  ...
  "datetime": "2022-03-20T04:45:00Z"
  ...
}
```

Example: Temporal extent

```
"properties": {
  ...
  "start_datetime": "2022-03-20T04:45:00Z",
  "end_datetime": "2022-03-22T04:45:00Z"
  ...
}
```

Example: No temporal description

```
"properties": {
  "datetime": null,
  ...
}
```

Requirement 10	/req/core/temporal
A	A WNM SHALL provide a temporal description by either a <code>properties.datetime</code> property or both the <code>properties.start_datetime</code> and <code>properties.end_datetime</code> properties.
B	The temporal description SHALL be in RFC3339 format.
C	The temporal description SHALL be in UTC.
D	The temporal description SHALL be set to <code>null</code> (using only <code>properties.datetime</code>) when a temporal description cannot be derived.

1.14 Properties / Cache

Core data, by default, is cached by Global Cache services, as described in the *Guide to WIS*, Volume II.

However, a data producer can use the `properties.cache` value to request Global Cache services to not cache their `core` data granule.

Example: Specifying data not to be cached

```
"properties": {
  "cache": false,
  ...
}
```

Permission 4	/per/core/cache
A	A WNM MAY specify, via the <code>properties.cache</code> property, whether the data should be cached .

1.15 Properties / Integrity

For data verification, it is recommended that data integrity information to be included via the `integrity` property. Providing this information will allow data consumers to ensure that a given data granule has not been corrupted during download.

The `method` property provides a format of the hashing method used to enable an integrity check of the data. The preferred values are `sha256`, `sha384`, `sha512`, `sha3-256`, `sha3-384`, and `sha3-512`.

The `value` property provides the result of the hashing method in base64 encoding.

Example:

```
"properties": {
  ...
  "integrity": {
    "method": "sha512",
    "value":
"CPvTLiOfYRgfL3YNF/KKElwamwvLQwnzd96VnF2WoYuuH+hVIbwFSPQHhd/qa/fNVUBckviC5/HZs3Nx2jXEs
A=="
  }
  ...
}
```

Recommendation 4	/rec/core/integrity
------------------	---------------------

A	A WNM SHOULD provide a <code>properties.integrity</code> property, consisting of a <code>method</code> property identifying the hashing method (<code>sha256</code> , <code>sha384</code> , <code>sha512</code> , <code>sha3-256</code> , <code>sha3-384</code> , <code>sha3-512</code>) and a <code>value</code> property of the hashing result, when it can be easily derived.
---	--

1.16 Properties / Content

The `content` property allows for the inclusion of data in the notification message when the length of the data, once encoded, is smaller than 4 096 bytes.

The `encoding` property provides the character encoding of the data (`UTF-8`, `base64` or `gzip`). The `gzip` encoding means that the data are compressed using the algorithm defined in [RFC1952](#) and consequently converted to text using Base64 encoding.

The `value` property provides the data in accordance with the `encoding` property.

The `size` property provides the size, in bytes, of the data in its original unencoded form, and therefore this value shall not be directly compared with the size limit.

Example:

```
"properties": {
  ...
  "content": {
    "encoding": "utf-8",
    "value": "encoded bytes from the file",
    "size": 457
  }
  ...
}
```

Requirement 11	/req/core/content
A	For data whose resulting size in the encoded form is greater than 4 096 bytes, notifications SHALL NOT provide the data inline via <code>properties.content.value</code> . Note that the encoding may either enlarge the data size (for example, when binary data, such as BUFR, is Base64 encoded), or reduce the size (for example, when XML data are compressed with gzip).

Recommendation 5	/rec/core/content
A	A WNM SHOULD provide the <code>content</code> property, consisting of an <code>encoding</code> property (either <code>utf-8</code> , <code>base64</code> or <code>gzip</code>), a <code>value</code> property of the data and a <code>size</code> property with the length of the data.

Permission 5	/per/core/content
--------------	-------------------

A	For data whose resulting size (after possible compression) is less than 4 096 bytes, notifications MAY provide the data inline via <code>properties.content.value</code> .
---	--

1.17 Links

The `links` array property consists of one or more objects providing URLs to access data.

Each link object provides:

- An `href` property with a fully qualified link to access the data;
- A `rel` property providing an [IANA link relation](#) or [WIS link type](#) describing the relationship between the link and the message;
- A `type` property providing the media type of the data;
- A `length` property providing the length (in bytes) indicating the size of the data;
- A `security` property providing a description of the access control mechanism applied (for example, recommended data with restrictions).

Links are used to communicate new data or metadata notifications. Links can also communicate when data or metadata have been deleted or invalidated.

Example: Canonical link

```
"links": [{
  "href": "https://example.org/data/4Pubsub/92c557ef-d28e-4713-91af-2e2e7be6f8ab.bufr4",
  "rel": "canonical",
  "type": "application/bufr"
}]
```

Example: Multiple links

```
"links": [{
  "href": "https://example.org/data/4Pubsub/92c557ef-d28e-4713-91af-2e2e7be6f8ab.bufr4",
  "rel": "canonical",
  "type": "application/bufr"
}, {
  "href": "https://example.org/oapi/collections/my-dataset/items/my-data-granule",
  "rel": "item",
  "type": "application/json"
}]
```

Requirement 12	/req/core/links
A	A WNM SHALL provide a <code>links</code> array property.

B	The links array property SHALL contain at least one link with, at a minimum, the href and rel properties.
C	The links for core data SHALL NOT require further action in order to download the resource.
D	The links SHALL be HTTP, HTTPS, FTP or SFTP.
E	For new data and metadata notifications, the links array property SHALL provide exactly one link with an IANA link relation of canonical to clearly identify the preferred access link.
F	For data or metadata update notifications, the links array property SHALL provide exactly one link with a link relation of update to clearly identify the preferred access link.
G	For data or metadata deletions, the links array property SHALL provide exactly one link with a link relation of deletion to clearly identify data which has been deleted or removed.
H	For metadata deletions, the properties.metadata_id property SHALL be specified to support deletion of a WCMP2 record from the Global Discovery Catalogue.
I	The links array property SHALL contain only one link with a link relation value of canonical , update , or deletion . These link relations SHALL NOT be combined within a single notification.

Recommendation 6	/rec/core/links
A	A WNM SHOULD provide links using secure protocols such as HTTPS and SFTP, with HTTPS being the preferred option.
B	The links array property SHOULD provide a length property to communicate the size of a given data download in advance of a data download workflow, when the size of the data is known or can be easily derived.
C	The link relation of deletion SHOULD NOT be used for communicating a rolling data archive.

Permission 6	/per/core/links
A	A WNM links array property MAY provide link objects which reference APIs or Web Accessible Folders (WAF).
B	In addition to one of the links required for WIS2 data and/or metadata publication, the links array property MAY provide additional link objects as necessary.

1.17.1 Access control

For recommended data, WNM links may also provide links to resources that implement access control in support of authentication and authorization. In secure data use cases, a user needs to be able to detect access-controlled data as part of data discovery and evaluation. The example

demonstrates how to express access control using HTTP Basic authentication for a given data access service.

Example: Access controlled link

```
"links": [{
  "rel": "data",
  "type": "application/json",
  "title": "link to WAF endpoint",
  "href": "https://example.org/data/secure-data",
  "security": {
    "default": {
      "type": "http",
      "scheme": "basic",
      "description": "Please contact the data provider for accessing this secured
resource."
    }
  }
}]
```

1.18 Additional properties

A WIS2 Notification Message can be extended as required for organizational purposes by adding properties (of any type) in the message. Additional properties do not render a notification non-compliant with the provisions of this specification.

Example:

```
"properties": {
  ...
  "_comment": {
    "validationErrors": [
      "error 1",
      "error 2"
    ]
  }
  ...
}
```

Example, based on WCMP2 client-side filtering:

```
"properties": {
  ...
  "wigos_station_identifier": "0-20000-0-71628"
  ...
}
```

Permission 7	/per/core/additional_properties
A	A WNM MAY provide additional properties of any type in any part of the document as needed.

2. WIS2 notification message resources

2.1 WMO Codes Registry

- <http://codes.wmo.int/wis/link-type>

2.2 WMO schemas server

Validation schemas, examples and other resources are published at <https://schemas.wmo.int/wnm>.

Annex A: Conformance Class Abstract Test Suite (Normative)

Conformance Class: Core

label

<http://wis.wmo.int/spec/wnm/1/conf/core>

subject

Requirements Class "core"

classification

Target Type:Notification Metadata

Message size

label

/conf/core/message_size

subject

/req/core/message_size

test-purpose

Validate that a WNM has a valid message size.

Check that the size of the complete WNM does not exceed 8192 bytes.

Validation

label

/conf/core/validation

subject

/req/core/validation

test-purpose

Validate that a WNM is valid to the authoritative WNM schema.

Run JSON Schema validation on the WNM against the WNM authoritative schema.

Identifier

label

/conf/core/identifier

subject

/req/core/identifier

test-purpose

Validate that a WNM has a valid identifier.

Check for the existence of an **id** property in the WNM.

Check that the **id** property is a valid UUID.

Conformance

label

/conf/core/conformance

subject

/req/core/conformance

test-purpose

Validate that a WNM notification provides valid conformance information.

Check for the existence of a **conformsTo** property in the WNM notification.

In the WNM notification's **conformsTo** array property, check that ONE of the values is equal to <http://wis.wmo.int/spec/wnm/1/conf/core>.

Version

label

/conf/core/identifier

subject

/req/core/identifier

test-purpose

Validate that a WNM has a valid version.

Check for the existence of an **version** property in the WNM.

Check that the **id** property is equal to **v04**.

Geometry

label

/conf/core/geometry

subject

/req/core/geometry

test-purpose

Validate that a WNM provides a valid geometry property.

Check for the existence of one **geometry** property in the WNM.

Check that all **geometry.coordinates** value data types are integers or floats.

Check that **geometry.coordinates** longitudinal values are between -180 and 180.

Check that **geometry.coordinates** latitudinal values are between -90 and 90.

Check that **geometry** property is a valid GeoJSON geometry.

Properties / Publication Time

label

/conf/core/pubtime

subject

/req/core/pubtime

test-purpose

Validate that a WNM has a valid pubtime.

Check for the existence of an **properties.pubtime** property.

Check that the **properties.pubtime** property is in RFC3339 format.

Check that the **properties.pubtime** property is in the UTC timezone.

Properties / Data Identification

label

/conf/core/data_id

subject

/req/core/data_id

test-purpose

Validate that a WNM has a valid data_id.

Check for the existence of an `data_id` property.

Properties / Temporal description

label

/conf/core/temporal

subject

/req/core/temporal

test-purpose

Validate that a WNM provides a valid temporal description.

Check for the existence of one `properties.datetime` or both `properties.start_datetime` and `properties.end_datetime`.

Check that the any of the `properties.datetime` or both `properties.start_datetime` and `properties.end_datetime` properties are in RFC3339 format.

Check that the any of the `properties.datetime` or both `properties.start_datetime` and `properties.end_datetime` properties are in the UTC timezone.

Properties / Global Cache

This requirement is not applicable to ATS testing.

Links

label

/conf/core/links

subject

/req/core/links

test-purpose

Validate that a WNM provides a link array property.

Check for the existence of a single **links** array property in the WNM.

Check that the **links** array property provides a minimum of one link object.

For each link object, check that the **href** property contains a valid protocol scheme of one of 'http', 'https', 'ftp', 'sftp'.

Check that the **links** array property contains one link object with a **rel** property with one of the values **canonical**, **update**, **deletion** as well as an **href** property.

Annex B: Schemas (Normative)

NOTE

The schema document will only be published on <https://schemas.wmo.int/wnm/1.1.0> once the standard has been approved.

WIS2 Notification Message Schema

```
$schema: https://json-schema.org/draft/2020-12/schema
$id: https://raw.githubusercontent.com/wmo-im/wis2-notification-
message/main/schemas/notificationMessageGeoJSON.yaml
title: WIS2 Notification Message Encoding
description: WIS2 Notification Message Encoding

type: object
properties:
  id:
    type: string
    format: uuid
    description: UUID (RFC4122) - Guarantee uniqueness of the message over (at least)
a 24h period.
  conformsTo:
    type: array
    contains:
      const: http://wis.wmo.int/spec/wnm/1/conf/core
  version:
    type: string
    description: DEPRECATED, please use `conformsTo`.
    const: v04
  type:
    type: string
    enum:
      - Feature
  geometry:
    oneOf:
      - enum:
          - null
      - $ref: 'https://raw.githubusercontent.com/opengeospatial/ogcapi-
features/master/core/openapi/schemas/pointGeoJSON.yaml'
      - $ref: 'https://raw.githubusercontent.com/opengeospatial/ogcapi-
features/master/core/openapi/schemas/polygonGeoJSON.yaml'
  properties:
    type: object
    properties:
      pubtime:
        type: string
        format: date-time
        description: |
          Identifies the date/time of when the file was posted/published, in RFC3339
```

format.

The publication date/time is critical for subscribers to prevent message loss by knowing

their lag (how far behind the publisher they are).

data_id:

type: string

description: |

Unique identifier of the data as defined by the data producer.

Data producers SHOULD NOT use an opaque id, but something meaningful to support client side filtering.

metadata_id:

type: string

description: |

Identifier for associated discovery metadata record to which the notification applies to.

The identification SHOULD be from a WCMP2 discovery metadata record identifier from the WIS2

Global Discovery Catalogue. Identifiers based on external catalogue systems MAY be articulated as additional links.

producer:

type: string

description: |

Identifies the provider that initially captured and processed the source data, in support of data distribution on behalf of other Members.

global-cache:

type: string

description: |

Centre identifier of Global Cache Service producing the notification.

datetime:

type:

- string

- null

format: date-time

description: Identifies the date/time of the data being published, in RFC3339 format.

start_datetime:

type: string

format: date-time

description: Identifies the start date/time date of the data being published, in RFC3339 format.

end_datetime:

type: string

format: date-time

description: Identifies the end date/time date of the data being published, in RFC3339 format.

cache:

type: boolean

description: |

Whether the data in the notification should be cached (if not specified, the default value is 'true').

When set to 'false', WIS2 Global Cache services do not cache the canonical

```

link, and publish the
    notification with an unmodified canonical link (which points back to the
endpoint as specified by the data producer).
    The notification is always published by the Global Cache regardless to the
`cache` topic.
    default: true
    integrity:
        type: object
        description: Specifies a checksum to be applied to the data to ensure that the
download is accurate.
        properties:
            method:
                type: string
                description: |
                    A specific set of methods for calculating the checksum algorithms:
                    * ``sha256``: the Secure Hash Algorithm 2, 256 bits, value is base64
encoded.
                    * ``sha384``: the Secure Hash Algorithm 2, 384 bits, value is base64
encoded.
                    * ``sha512``: the Secure Hash Algorithm 2, 512 bits, value is base64
encoded.
                    * ``sha3-256``: the Secure Hash Algorithm 3, 256 bits, value is base64
encoded.
                    * ``sha3-384``: the Secure Hash Algorithm 3, 384 bits, value is base64
encoded.
                    * ``sha3-512``: the Secure Hash Algorithm 3, 512 bits, value is base64
encoded.
            enum:
                - sha256
                - sha384
                - sha512
                - sha3-256
                - sha3-384
                - sha3-512
        value:
            type: string
            contentEncoding: base64
            description: Checksum value.
    required:
        - method
        - value
    content:
        type: object
        description: Used to embed small products inline within the message.
        properties:
            encoding:
                type: string
                description: Encoding of content
            enum:
                - utf-8
                - base64

```

```

    - gzip
  size:
    type: integer
    maximum: 4096
    description: |
      Number of bytes contained in the file. Together with the ``integrity``
      property, it provides additional assurance that file content was accurately received.
      Note that the limit takes into account the data encoding used, including
      data compression (for example `gzip`).
    value:
      type: string
      description: The inline content of the file.
      maxLength: 4096
  required:
    - encoding
    - size
    - value
  required:
    - pubtime
    - data_id
  oneOf:
    - allOf:
      - required:
          - start_datetime
          - end_datetime
    - allOf:
      - required:
          - datetime

  links:
    type: array
    minItems: 1
    items:
      allOf:
        - $ref: 'https://raw.githubusercontent.com/opengeospatial/ogcapi-
          features/master/core/openapi/schemas/link.yaml'
        - properties:
            security:
              type: object
              $ref: https://raw.githubusercontent.com/OAI/OpenAPI-
                Specification/3.1.0/schemas/v3.0/schema.yaml#/definitions/Components/properties/securitySchemes

  oneOf:
    - required:
        - id
        - conformsTo
        - type
        - geometry
        - properties
        - links

```

- required:
 - id
 - version
 - type
 - geometry
 - properties
 - links

example:

\$ref: <https://raw.githubusercontent.com/wmo-im/wis2-notification-message/main/examples/example1.json>

Annex C: Examples (Informative)

WIS2 Notification Message Examples

Example: single observation including meaningful geometry, observation datetime and content

```
{
  "id": "31e9d66a-cd83-4174-9429-b932f1abe1be",
  "conformsTo": [
    "http://wis.wmo.int/spec/wnm/1/conf/core"
  ],
  "type": "Feature",
  "geometry": {
    "type": "Point",
    "coordinates": [
      6.146255135536194,
      46.223296618227444
    ]
  },
  "properties": {
    "pubtime": "2022-03-20T04:50:18Z",
    "datetime": "2022-03-20T04:45:00Z",
    "integrity": {
      "method": "sha512",
      "value": "A2KNxvks...S8qfSCw=="
    },
    "data_id": "dataset/123/data-granule/UANT01_CWAO_200445___15103.bufr4",
    "metadata_id": "urn:wmo:md:ca-eccc-msc:observations.swob",
    "content": {
      "encoding": "utf-8",
      "value": "encoded bytes from the file",
      "size": 457
    }
  },
  "links": [
    {
      "href": "https://example.org/data/4Pubsub/92c557ef-d28e-4713-91af-2e2e7be6f8ab.bufr4",
      "rel": "canonical",
      "type": "application/bufr"
    },
    {
      "href": "https://example.org/oapi/collections/my-dataset/items/my-data-granule",
      "rel": "item",
      "type": "application/json"
    }
  ]
}
```


Example: NWP product including polygon **geometry**, **start_datetime** and **end_datetime**. The polygon is an array with 5 elements. The first (and last) elements are most southwestern point of the rectangle using longitude and latitude and followed by other coordinates in a clockwise direction.

```
{
  "id": "31e9d66a-cd83-4174-9429-b932f1abe1be",
  "conformsTo": [
    "http://wis.wmo.int/spec/wnm/1/conf/core"
  ],
  "type": "Feature",
  "geometry": {
    "type": "Polygon",
    "coordinates": [
      [
        [
          -7.75,
          40.43
        ],
        [
          -7.75,
          78.46
        ],
        [
          71.91,
          78.46
        ],
        [
          71.91,
          40.43
        ],
        [
          -7.75,
          40.43
        ]
      ]
    ]
  },
  "properties": {
    "pubtime": "2022-03-20T04:50:18Z",
    "start_datetime": "2022-03-20T00:06:00Z",
    "end_datetime": "2022-03-20T00:12:00Z",
    "integrity": {
      "method": "sha512",
      "value": "A2KNxvks...S8qfSCw=="
    },
    "data_id": "dataset/4546/abcdef-abcd-abcd-abcd-abcdefabcd",
    "metadata_id": "urn:wmo:md:fr-meteo-france:gap123"
  },
  "links": [
    {
      "href": "https://example.org/data/nwp/92c557ef-d28e-4713-91af-
```

```

2e2e7be6f8ab.grib",
    "rel": "canonical",
    "type": "application/grib"
  }
]
}

```

Example: minimal viable message compliant to the specification

```

{
  "id": "31e9d66a-cd83-4174-9429-b932f1abcdef",
  "conformsTo": [
    "http://wis.wmo.int/spec/wnm/1/conf/core"
  ],
  "type": "Feature",
  "geometry": null,
  "properties": {
    "pubtime": "2022-11-20T16:40:37Z",
    "datetime": "2022-11-11T11:11:11Z",
    "data_id": "data/data-123/items/abcdefab-abcd-1234-5678-0123456789ab",
    "metadata_id": "urn:wmo:md:test-nmhs-xyz:some_dataset",
    "cache": false
  },
  "links": [
    {
      "href": "https://example.org/92c557ef-d28e-4713-91af-2e2e7be6f8ab.txt",
      "rel": "canonical"
    }
  ]
}

```

Example: notification of the deletion of a data granule

```

{
  "id": "31e9d66a-cd83-4174-9429-b932f1abcdef",
  "conformsTo": [
    "http://wis.wmo.int/spec/wnm/1/conf/core"
  ],
  "type": "Feature",
  "geometry": null,
  "properties": {
    "pubtime": "2022-12-22T16:40:37Z",
    "datetime": "2022-11-11T11:11:11Z",
    "data_id": "data/data-123/items/abcdefab-abcd-1234-5678-0123456789ab",
    "metadata_id": "urn:wmo:md:test-nmhs-xyz:some_dataset",
    "cache": false
  },
  "links": [
    {
      "href": "https://example.org/92c557ef-d28e-4713-91af-2e2e7be6f8ab.txt",

```

```
    "rel": "deletion"
  }
]
```

Annex D: Bibliography

- W3C/OGC: Spatial Data on the Web Best Practices, W3C Working Group Note 28 September 2017, <https://www.w3.org/TR/sdw-bp>
- W3C: Data on the Web Best Practices, W3C Recommendation 31 January 2017, <https://www.w3.org/TR/dwbp>
- IANA: Link Relation Types, <https://www.iana.org/assignments/link-relations/link-relations.xml>
- ISO8601: Date and time — Representations for information interchange, <https://www.iso.org/standard/70907.html>
- NIMA: Department of Defense World Geodetic System 1984: Its Definition and Relationships with Local Geodetic Systems", Third Edition 1984, <https://apps.dtic.mil/sti/citations/ADA280358>

Annex E: Revision History

Date	Release	Proposed by	Primary clauses modified	Approved by
June 2024	1.0.0	ET-Metadata	Clause 7	EC-78
November 2024	1.1.0	ET-Metadata	Clause 7 (See GitHub)	Fast-track 2024-2
February 2026	1.2.0	ET-Metadata	Clause 7 (See GitHub)	Fast-track 2025-2
June 2026	1.3.0	ET-Metadata	Clause 7 (See GitHub)	Fast-track 2026-1